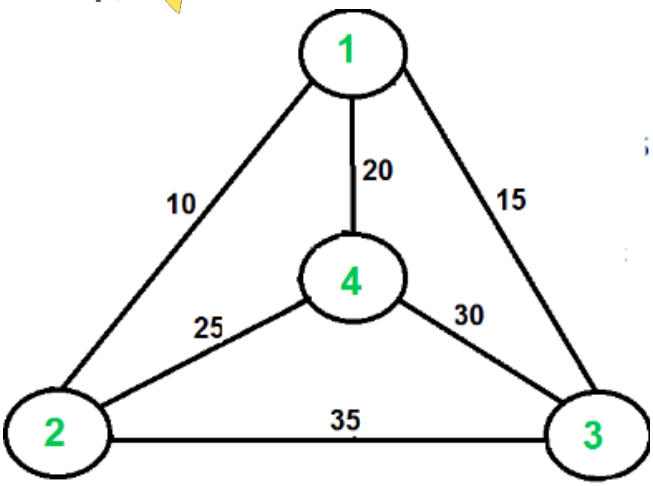
1.1.13. Travelling... 10:45

Given a set of cities and the distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point.

Example:



A TSP tour in the graph is 1-2-4-3-1. The cost of the tour is 10+25+30+15, which is 80

Input Format:

- The first line contains an integer n, representing the number of cities.
- The following n lines contain the cost matrix of size $n \times n$. Each line contains n integers separated by spaces, representing the cost of traveling from one city to another. If there is no direct connection between two cities, -1 is used to represent infinity.

Output Format:

- The output is a single integer representing the optimal cost of the traveling salesman problem.

Explorer

CTC30966.c

Submit

Debugger

```c
#include <stdio.h>
#include <stdlib.h>

#define INF 1e9

int n;
int matrix[20][20];
int dp[1 << 20][20];

int min(int a, int b) {
    return (a < b) ? a : b;
}

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    int num_states = 1 << n;
    for (int i = 0; i < num_states; i++) {
        for (int j = 0; j < n; j++) {
            dp[i][j] = INF;
        }
    }

    dp[1][0] = 0;

    for (int mask = 1;
```

‹ Prev Reset Submit

Sample Test Cases +

```
mask++) {
        for (int u = 0; u < n; u++) {
            if (dp[mask][u] == INF) continue;

            for (int v = 0; v < n; v++) {
                if (!(mask & (1 << v)) && matrix[u][v] != -1) {

dp[mask | (1 << v)][v] = min(dp[mask | (1 << v)][v], dp[mask][u] + matrix[u][v]);
                }
            }
        }
    }

    int ans = INF;
    for (int v = 0; v < n; v++) {
        if (matrix[v][0] != -1 && dp[num_states - 1][v] != INF) {
            ans = min(ans, dp[num_states - 1][v] + matrix[v][0]);
        }
    }

    if (ans == INF) {
        printf("-1\n");
    } else {
        printf("%d\n", ans);
    }

    return 0;
}
```